

Cybersecurity — Exam Practice Sheets

Security of Mobile and Web-based Applications · SoSe 2026

Prof. Dr. Thomas Hutzelmann · TH Ingolstadt · source slides CC BY 4.0

How to use

Each sheet has the questions on the top half and model answers on the bottom half. **Fold the page along the dashed line** to hide the answers while you practice, then unfold to self-correct. Score each part using the ____/N fields, total per chapter, and look up your grade below. Slide citations [p.NNN] point back to the source deck.

Your score per chapter

#	Chapter	Max	Mine	Grade
1	Foundations	10 P		
2	Requirements Engineering	18 P		
3	Human Security Factors	23 P		
4	Empirical Security Research	9 P		
5	Injection Attacks	28 P		
6	Cryptography	15 P		
7	Authorization	16 P		
Σ	Total	119 P		

Grading key (% of max points)

≥ %	Grade
95	1.0 — sehr gut
90	1.3
85	1.7
80	2.0 — gut
75	2.3
70	2.7
65	3.0 — befriedigend
60	3.3
55	3.7
50	4.0 — ausreichend
< 50	5.0 — nicht bestanden

Point values are an estimate for self-assessment; the real exam weighting is not published.

Answer in the space below each question; fold along the line to check.

- a) Define the term “(Cyber)Security”. (2 P)
- b) Both attackers and defenders look for vulnerabilities in applications. Explain why this task is generally easier for attackers than for defenders. (3 P)
- c) Explain briefly why the attacks demonstrated and carried out in this course are not punishable under German law. (2 P)
- d) Evaluate the following from the perspective of German law: while developing a campus app, a student discovers a vulnerability in the university’s API and, to prove it, copies parts of the database with all students’ data to his own laptop. (3 P)

ANSWERS (your score: _____ / 10 P)

 fold here

- a) Cybersecurity is the **protection of computer systems and networks from attacks by malicious actors** that may result in unauthorized information disclosure, theft of or damage to hardware, software or data, as well as from the disruption or misdirection of the services they provide. It is a **“Not”-property**: assuring the **absence of unwanted functionality**, not an explicitly programmed feature.
 ✓ **Likely graded:** the protection idea + naming the protected goals (data/systems/services) against malicious actors; bonus: security as a “Not”-property. [p.109, 112] _____/2
- b) The **defender must find and close every single weakness** — proving the **absence** of unwanted behaviour requires checking **all** possible system interactions. The **attacker only needs to find and exploit one single vulnerability**. The work is therefore asymmetric: complete coverage vs. a single hole.
 ✓ **Likely graded:** the asymmetry — defender must cover **all** weaknesses, attacker needs only **one**; absence of unwanted behaviour is hard to prove. [p.112, 113] _____/3
- c) The attacks are performed **with authorization** on systems provided for exactly this purpose (your own machine / the supplied CTF containers), so access is **not “unbefugt”** (unauthorized) as required by §202a StGB. In line with Germany’s highest courts, the **purpose** is teaching/training, research and curiosity — not the criminal purpose the “hacker paragraph” (§202c) targets.
 ✓ **Likely graded:** authorized access on provided/own systems (not “unbefugt”) + legitimate purpose (teaching/research) recognised by the courts. [p.120, 123] _____/2
- d) **Punishable.** By exploiting the vulnerability he **circumvents the access protection** and **obtains access to data not intended for him** (all students’ data) without authorization — this fulfils **§202a StGB (Ausspähen von Daten)**, up to 3 years. He has no pentest contract, so the access is “unbefugt”, and copying the data exceeds what proving the flaw needs. Correct behaviour would be **responsible disclosure** (e.g. via the CCC) **without exfiltrating real data**.
 ✓ **Likely graded:** §202a applies (unauthorized access to specially-protected data not intended for him by overcoming protection); no authorization; copying real data is unnecessary; name responsible disclosure as the lawful alternative. [p.120, 124] _____/3

Answer in the space below each question; fold along the line to check.

- a) Define the security attribute “Availability”. (2 P)

- b) Define the security attribute “Integrity” for data and systems. (2 P)

- c) Give an example of the security principle “Economy of mechanism”. (1 P)

- d) Give an example of the security principle “Defense in depth”. (1 P)

- e) Define the security principle “Least Common Mechanism”. (2 P)

ANSWERS (your score: _____ / 8 P)

 fold here

- a) **Availability is ensuring timely and reliable access to and use of information — i.e. preventing absences and downtimes.** Exemplary means: defining a maximum amount of resource requests (cloud scaling) and having **backup and recovery plans**.
 ✓ **Likely graded:** timely + reliable access/use of information; preventing downtime. [p.216, 213] _____/2
- b) **Integrity means guarding against improper modification or destruction of information, and includes ensuring non-repudiation and authenticity. It prevents unintentional alterations;** means include defining who may change what and using **checksums / digital signatures** to detect changes.
 ✓ **Likely graded:** guarding against improper/unintended modification or destruction; includes non-repudiation + authenticity. [p.215, 213] _____/2
- c) **Hardware security modules (HSMs), or Let’s Encrypt SSL certificates — keeping the design as simple and small as possible.**
 ✓ **Likely graded:** one valid example (HSMs or Let’s Encrypt). [p.234] _____/1
- d) **Two-factor authentication / TANs for sensitive actions, or firewalls plus demilitarized zones — multiple independent layers of defense.**
 ✓ **Likely graded:** one valid example (2FA/TAN or firewall + DMZ). [p.256] _____/1
- e) **Minimize artifacts shared among different applications and users.** Rationale: a single vulnerability then **affects only a single application/user**, and diversity complicates large-scale attacks (e.g. a different key per permission within a session).
 ✓ **Likely graded:** minimize shared artifacts/mechanisms between users/apps; limits blast radius of one flaw. [p.246] _____/2

Answer in the space below each question; fold along the line to check.

- f) Define the security principle “Psychological Acceptability”. (2 P)
- g) Name one argument for and one argument against the application of the security principle “Open Design”. (2 P)
- h) Is it possible to satisfy the security attribute “Non-repudiation” without “Accountability” in the same activity? Justify your answer using the definitions of the two terms. (3 P)
- i) Is it possible to satisfy both the security attribute “Privacy” and “Anonymity” for the same data at the same time? Justify your answer using the definitions of the two terms. (3 P)

ANSWERS (your score: _____ / 10 P)

 fold here

- f) **Match the security mechanism to the user’s mental image.** Rationale: we want users to actually **use and prefer secure systems**; otherwise legitimate users **work around** the security measures to get their task done.
 ✓ **Likely graded:** fit mechanism to user’s mental model; else users circumvent it. [p.248] _____/2
- g) **Open Design** = share implementation details with others, even the attacker. **For:** it enables **reviews and white-box analyses** and re-use of designs (cf. **Kerckhoffs**: a cipher should not require secrecy; **Linus’ Law**: “given enough eyeballs, all bugs are shallow”). **Against:** the “many eyeballs” claim is contested — Glass lists it as **Fallacy 8**, and bugs can survive long in popular open code (e.g. **Heartbleed** in OpenSSL: introduced 2012, found 2014).
 ✓ **Likely graded:** one pro (review/white-box, Kerckhoffs/Linus’ Law) + one con (eyeballs fallacy / Heartbleed longevity). [p.240, 264, 265] _____/2
- h) **No. Non-repudiation** protects against an individual **falsely denying having performed a particular action**, i.e. one must be able to **determine whether a given individual took that action**. **Accountability** is the property of **being able to trace activities to the individuals** responsible. Determining that a specific person did the action (non-repudiation) already **requires tracing the action to that individual** — which is exactly accountability. So non-repudiation **presupposes** accountability and cannot be satisfied without it in the same activity.
 ✓ **Likely graded:** No + reasoning: non-repudiation requires attributing the action to a specific individual, which is what accountability provides, so it presupposes accountability. [p.224, 223] _____/3
- i) **Yes. Privacy** is a party’s **right to keep control over and confidentiality of information about itself**; **Anonymity** is the condition that an entity **can be recognized as distinct without enough identity information to link it to a known identity**. They target different aspects and are **complementary**: anonymity removes the **link** to the real identity, while privacy **controls/protects** the data that does exist. Concealing the identity link does not conflict with controlling the data — both can hold at once (e.g. published **grade averages only**, with strict retention limits).
 ✓ **Likely graded:** Yes + reasoning: anonymity hides the identity link, privacy controls/protects the data; different aspects, complementary, not mutually exclusive. [p.218, 219] _____/3

Answer in the space below each question; fold along the line to check.

- a) Explain why using readable **passphrases** consisting only of lowercase letters is generally considered a better alternative to passwords that contain a random mix of letters, numbers, and special characters. (3 P)
- b) Name two advantages of **passphrases** (natural-language words) compared to **passwords** (random strings that may include numbers and special characters). (2 P)
- c) A common requirement is that passwords be changed regularly, e.g. every 90 days. Assess this requirement regarding its impact on **security** and **user behavior**. (3 P)

ANSWERS (your score: _____ / 8 P)

 fold here

- a) Entropy grows **logarithmically with the alphabet size, but linearly with the length** ($H(\omega) = n \cdot \log_2 |\Omega|$). Adding character classes only enlarges the alphabet a little, whereas a long passphrase wins many characters of length — so a long lowercase phrase reaches **more entropy** than a short mixed password. It is also **memorable and easy to type**, so users do not fall back on weak optimisations: the simple “throwing-in-special-characters” passwords are exactly the **patterns/common words** that crackers (e.g. zxcvbn) expect, since users are **bad random number generators** and prefer characters they can type. A readable phrase that is long therefore beats a short random string in both real strength and usability.
 ✓ **Likely graded:** length beats alphabet size for entropy (linear vs. log) + passphrases are long → high entropy; usable/memorable so users do not pick weak predictable patterns crackers expect. [p.311, 313, 316] _____/3
- b) (1) They are **easier to memorise and to type**, since they use natural-language words a user can actually remember. (2) They are **generally longer**, and length contributes linearly to entropy, so they achieve **higher entropy / are harder to guess** while staying user-friendly.
 ✓ **Likely graded:** any two of: easier to remember; easier/faster to type; longer ⇒ more entropy / harder to guess. [p.316, 311] _____/2
- c) The **intended security gain**: if password hashes leak, forced rotation makes them useless to an attacker after a while. **But the gain is largely undermined by user behaviour**: users cope with frequent changes by using **patterns** to derive similar successor passwords (e.g. incrementing a number). An attacker who knows one old password can then **more easily guess the new ones** (documented by Zhang et al. on a campus SSO system). Forced rotation thus pushes users toward predictable, weaker passwords. **Conclusion (NIST 2024)**: do **not** require periodic changes; only force a change after **indicators of compromise**.
 ✓ **Likely graded:** name intended benefit (leaked hash becomes stale) BUT users react with predictable patterns/incremental passwords (Zhang campus study) → easier to guess; conclude NIST: change only on evidence of compromise. [p.314] _____/3

Answer in the space below each question; fold along the line to check.

- a) Scenario: after a wrong password the login screen shows a large red message: “Warning! AuthErr\#42. Contact phone support immediately (Mondays only, 7:00–7:30 a.m.). This error may result in criminal charges!” Below it a single “Agree” button. Name **two** guidelines for user-centric IT security that are violated, and briefly explain each violation. (4 P)
- b) Scenario: opening <https://example.org> the browser shows only a red box “Warning! TLS Error \#13.” — no other buttons or controls. Refer to a violated guideline for user-centric IT security, explain why it is violated, and give a specific proposal to fix it. (4 P)

ANSWERS (your score: _____ / 8 P)

 fold here

- a) **G1 — Understandability (open for all users):** the message AuthErr\#42 is a **short, cryptic error code** with no plain-language explanation; users are not security experts and cannot understand what went wrong or what to do. **G7 — Fearless system:** threatening “criminal charges!” **uses fear to motivate the user** and makes them anxious, instead of promoting a positive, cooperative attitude. (Also defensible: **G8 — educated reaction on user errors** — a cryptic message with no healing action and only an “Agree” button gives no step-by-step guidance for the simple error of mistyping a password.)
 ✓ **Likely graded:** two distinct guidelines named + matched to scenario: G1 (cryptic code, not understandable) and G7 (fear-mongering “criminal charges”); G8 (no helpful guidance/healing on a user error) also accepted. [p.320, 322, 329, 330] _____/4
- b) Violated: **G5 — Only informed decisions.** The user is **given no information** (just a cryptic code) and **no way to act** — yet is effectively forced to abandon the task without understanding the risk; the system neither provides the necessary information nor lets the user make an informed choice. (**G1 understandability is likewise violated by the cryptic code.**) **Proposal:** follow the slide’s example of **browser certificate warnings** — explain in plain language what is wrong (e.g. the site’s TLS certificate could not be verified, so the connection may be insecure) **and** offer real options: a “Go back to safety” button and an “Advanced / proceed anyway” option, so the user can make an informed decision.
 ✓ **Likely graded:** name G5 (no information + no possible decision) [G1 acceptable too]; concrete fix = plain-language explanation of the risk plus actionable buttons (back / proceed), like browser certificate warnings. [p.320, 326, 327] _____/4

Answer in the space below each question; fold along the line to check.

- a) The **Robin Sage** example shows social engineering on a social network to obtain contacts and sensitive information. Name **two** of **Cialdini's Principles of Persuasion** and describe how attackers apply each when creating a fake profile. (4 P)

- b) Outline a social engineering attack on a **university campus** based on one of Cialdini's Principles of Persuasion. Name the principle and describe the attack in one or two sentences. (3 P)

ANSWERS (your score: _____ / 7 P)

 fold here

- a) **Authority:** people tend to obey authority figures. The fake “Robin Sage” listed **high-status credentials** (a “Cyber threat analyst” at the Naval Network Warfare Command, MIT graduate, 10 years’ experience), so targets trusted and complied with the seemingly senior expert. **Consensus / social proof:** people conform to their group’s norms. After connecting with a few real professionals, the **visible list of mutual, reputable contacts** (executives at NSA, DOD, Global 500 firms) made the profile look legitimate, so others followed and accepted requests too. **(Also applicable: Liking — an attractive 25-year-old persona makes people more willing to be persuaded.)**

✓ **Likely graded:** two distinct Cialdini principles named + concrete Robin-Sage application each: e.g. Authority (fake elite credentials/job) and Consensus/social proof (chain of reputable mutual contacts); Liking/Scarcity etc. also valid if tied to the profile. [p.344, 345, 347] _____/4

- b) **Principle: Authority.** The attacker walks the campus **dressed as IT support staff** (badge/lanyard), tells a student their account shows a problem and that “the dean’s office needs it fixed now”, and asks them to log in or hand over their password on a prepared laptop — the assumed authority and urgency make the student comply. **(Alternatives: Reciprocity — hand out free USB sticks/coffee then ask a “small favour”; Liking — a friendly peer asks to “borrow” library/exam access.)**

✓ **Likely graded:** one principle clearly named + a coherent campus attack that actually uses it (e.g. Authority via fake IT/staff impersonation); brief and concrete. [p.344] _____/3

Answer in the space below each question; fold along the line to check.

- a) What are the benefits of recording vulnerabilities in a central, public database using unique identifiers? Name two aspects. (2 P)
- b) Why is it important to also collect weaknesses in a central, public vulnerability database and to link all registered vulnerabilities to one such weakness? Name two reasons. (2 P)
- c) What are the benefits of risk assessment (e.g., with CVSS) when dealing with vulnerabilities? Name two aspects. (2 P)
- d) Why do the first ten entries of the CWE Top 25 differ from the entries of the OWASP Top 10 from the same year? Name three reasons. (3 P)

ANSWERS (your score: _____ / 9 P)

 fold here

- a) Using a central public database like **CVE** with a **globally unique identifier** per vulnerability gives, e.g.: (1) **unambiguous reference** — everyone (vendors, CNAs, tools, researchers) can talk about exactly the same flaw without confusion, even across versions/products. (2) **Collect, enumerate and share knowledge** — a continuously growing, public record of known, documented and disclosed security bugs that anyone can **search and aggregate** (e.g. count CVEs per product, track growth over years). It also **enables a concrete reaction**: the entry names the affected software/version so users can **patch**, and it can be **linked** to a weakness (CWE) and a risk score (CVSS).
 ✓ **Likely graded**: two distinct benefits, e.g. unique/unambiguous reference shared by all parties + public collection enabling search/aggregation/statistics; bonus: names affected version so one can patch. [p.408, 410, 415] _____/2
- b) A **vulnerability** (CVE) is one concrete, exploitable instance in a specific version; a **weakness** (CWE) is the **underlying type/class** — the condition that **causes** vulnerabilities. Collecting weaknesses and linking each vulnerability to its weakness gives, e.g.: (1) it **describes the common root cause** of many otherwise separate CVEs, so you understand **why** the flaw happens, not just where. (2) It enables a **preventive, long-term reaction** — **awareness and training** to avoid the weakness in the future (a patch only fixes the single instance). It also turns the data into a **fixed, stable, organised list** you can **aggregate and rank** (e.g. count how often a CWE appears across CVEs → CWE Top 25).
 ✓ **Likely graded**: two distinct reasons, e.g. captures the shared root cause/origin of many CVEs + enables prevention via awareness/training (vs. only patching one instance); bonus: stable list allows aggregation/ranking. [p.407, 409, 410] _____/2
- c) Risk assessment turns Risk = Impact × Likelihood into a comparable measure; **CVSS** expresses this as a **quantitative base score 0–10** combining **Exploitability** (≈ likelihood) and **Impact**. Benefits, e.g.: (1) **Prioritisation** — a comparable score lets you decide where to spend limited resources (e.g. low risk → accept, high risk → fix) instead of treating all vulnerabilities equally. (2) **Standardised, transparent communication** — a shared metric and **vector string** lets different parties consistently quantify and justify severity (and refine it per organisation via temporal/environmental metrics). This counters both over- and under-estimating risk, which otherwise wastes resources or leaves painful residual risk.
 ✓ **Likely graded**: two distinct benefits, e.g. comparable score enables prioritisation / resource allocation (accept vs. fix) + standardised, justifiable severity shared across parties; bonus: avoids over-/under-estimation. [p.420, 426, 430] _____/2
- d) Although the two lists overlap strongly, they are **built differently**: (1) **Different scope** — the **OWASP Top 10 covers only web applications**, while the **CWE Top 25 is more general** and also includes **binary/memory vulnerabilities** (e.g. buffer overflows). (2) **Different data sources** — the CWE Top 25 is computed from **openly disclosed CVEs** in public databases, whereas the OWASP Top 10 also uses **non-publicly disclosed / proprietary data submitted by companies** plus an **expert survey** for future trends. (3) **Different methodology** — CWE Top 25 ranks by **Danger Score** = Frequency × Severity (avg CVSS) over mapped CVEs, while OWASP groups/refines by human experts and **differentiates the tooling used for discovery**; the **ranking thus differs per list and per version**. In short: **different definitions of scope lead to different results**.
 ✓ **Likely graded**: three distinct reasons, e.g. scope (web-only vs. general incl. binary) + data source (public CVEs vs. private submissions + survey) + methodology/abstraction (frequency×severity vs. expert grouping, tooling differentiation). [p.444, 446, 453] _____/3

Answer in the space below each question; fold along the line to check.

- a) Name the vulnerability that is checked for by entering the following string: `<script>alert(1);</script>` (1 P)
- b) You suspect a SQL Injection vulnerability in a text input field. Provide a string you would submit to test your suspicion and describe briefly which server behavior would confirm the vulnerability. (3 P)
- c) You suspect a Cross-Site-Scripting (XSS) vulnerability in a text input field. Name a string you could submit to test your suspicion and describe briefly which server behavior would confirm the vulnerability. (3 P)
- d) Classify the impact a SQL Injection vulnerability in a SELECT query could have on 1) confidentiality, 2) integrity and 3) availability of the data, and describe briefly a concrete action an attacker can take to cause each impact. (6 P)

ANSWERS (your score: _____ / 13 P)

 fold here

- a) **Cross-Site Scripting (XSS)** (CWE-79). The `<script>` tag is injected HTML that, if reflected unescaped into the page, makes the browser **execute the attacker's JavaScript**.
 ✓ **Likely graded:** naming **Cross-Site Scripting (XSS)**. [p.509, 510, 511] _____/1
- b) Submit a string that breaks out of the SQL string literal, e.g. a single ' or the classic ' OR 1=1 -- . **Confirmation:** a lone ' produces a **SQL syntax error** (e.g. an error page / DB error message), and ' OR 1=1 -- **returns more rows than expected** (e.g. logs you in / lists all records). Either shows the input is interpreted as **code, not data**.
 ✓ **Likely graded:** a payload with ' (and/or OR 1=1 --) + the confirming behavior: SQL error from the broken quote, or extra/all rows returned. [p.522, 525, 531] _____/3
- c) Submit `<script>alert(1);</script>` (or an event handler such as `<svg onload=alert(1)>`). **Confirmation:** the application **reflects/stores the string unescaped** into the page so the browser **runs the script** — i.e. the `alert(1)` popup appears. If instead it is returned as escaped text (`<script>`), it is **not** vulnerable.
 ✓ **Likely graded:** a `<script>/event-handler` payload + the confirming behavior: the script executes (popup) because the input is reflected unescaped into the page. [p.510, 511, 514] _____/3
- d) **1) Confidentiality:** attacker **reads data not intended for them** — e.g. append `UNION SELECT username, password FROM users` to dump the credential table. **2) Integrity:** attacker **alters data** — by **stacking** a second statement (; `UPDATE users SET ... / ; DELETE FROM ...`) the query is misused to modify or delete records. **3) Availability:** attacker **exhausts server resources** — e.g. inject `OR SLEEP(...)` or a very heavy/expensive query so the DB thread is blocked, causing a denial of service.
 ✓ **Likely graded:** each of the three attributes named with one concrete action: read foreign data (UNION SELECT), modify/delete data (stacked UPDATE/DELETE), exhaust resources (SLEEP / heavy query → DoS). [p.524, 525, 527] _____/6

After a security incident in a web app, the database logs show many recent database queries like the following:

```
SELECT * FROM session WHERE key = 'abc' OR SLEEP(4*7*24*60*60) --';
```

(Note: abc is a valid session key. This is not relevant for the answers.)

Answer in the space below each question; fold along the line to check.

- Mark the complete payload inserted by the attacker. (2 P)
- Name the exact type of attack that was executed here. (2 P)
- Describe briefly what this single request and what numerous such requests in a short period of time can cause on the server. (3 P)
- Name the security attribute that these queries violate. (1 P)
- Describe briefly two additional, distinct damages an attacker can inflict based on the underlying vulnerability. (4 P)
- Describe briefly how this vulnerability can be fixed in the web application so such attacks are no longer possible. (3 P)

ANSWERS (your score: _____ / 15 P)

 fold here

- The injected part after the legitimate value 'abc', namely: `OR SLEEP(4*7*24*60*60) --` (a space follows the `--`). It adds an always-evaluated condition and comments out the rest of the original query.
✓ **Likely graded:** identifying `OR SLEEP(4*7*24*60*60) --` (the part after 'abc') as the payload, including the trailing `--` comment. [p.522, 525] _____/2
- A **(time-based) Blind SQL Injection**. The query has no visible output (a session lookup), so the attacker abuses the **timing side-effect** of `SLEEP()`.
✓ **Likely graded:** naming **SQL Injection**; full marks for **time-based blind SQL injection**. [p.525, 527, 530] _____/2
- `SLEEP(4*7*24*60*60) = 4*7*24*60*60 seconds ≈ 4 weeks`. A single request blocks one DB connection/thread asleep for 4 weeks. **Many** such requests in a short time **occupy all available connections/threads, exhausting the server's resources** → the database becomes unresponsive — a **denial of service (DoS)**.
✓ **Likely graded:** single request = one thread blocked 4 weeks (4·7·24·60·60 s); many requests = resource/connection exhaustion → DoS / server unresponsive. [p.527, 532] _____/3
- Availability**.
✓ **Likely graded:** naming **Availability**. [p.524, 527] _____/1
- The underlying flaw is **SQL injection** (input used as code), so beyond DoS the attacker can also: **1) harm confidentiality** — read/dump data not intended for them, e.g. extract the whole database via `UNION SELECT` or bit-by-bit through blind techniques. **2) harm integrity** — modify or destroy data, e.g. `stack ; UPDATE/DELETE/DROP` to change records or drop tables.
✓ **Likely graded:** two distinct damages from the same SQLi flaw: confidentiality (read/dump foreign data) and integrity (modify/delete/drop data). [p.524, 525] _____/4
- Use **Prepared Statements / parameterized queries**: the query **structure is sent and “compiled” first**, then the data is **bound** separately (e.g. `prepare("... WHERE key = ?") + bind_param`). Because **code and data are strictly separated**, input can never change the query's structure, so injection is impossible. (Manual escaping is insufficient — it only protects quoted strings, not other datatypes.)
✓ **Likely graded:** naming **Prepared Statements / parameterized queries** and the reason: separate query structure from data so input is never interpreted as code. [p.534, 536] _____/3

Answer in the space below each question; fold along the line to check.

- a) Describe briefly the difference between point-to-point encryption (P2PE) and end-to-end encryption (E2EE). (2 P)
- b) Explain why modern cryptographic protocols use a combination of symmetric and asymmetric encryption rather than 1) symmetric encryption alone or 2) asymmetric encryption alone. (3 P)
- c) Alice opens the website <https://example.org> using the current version of TLS. List all cryptographic keys required to establish a secure connection and describe their purpose. (4 P)
- d) Alice sends an end-to-end encrypted email with a large attachment to Bob (e.g. using S/MIME or PGP). Name the necessary keys and describe how such a protocol ensures that: (1) the email can be sent and read quickly; (2) no one but Bob can read the message; (3) Bob received the complete, unchanged message from Alice; (4) the email actually arrives and was read. (6 P)

ANSWERS (your score: _____ / 15 P)



fold here

- a) **P2PE** secures the link only between two points on the network (e.g. client-server), so **every intermediate** (server/relay) can read the content; security depends on trusting those intermediates. Easier to deploy and scale — e.g. TLS, Teams. **E2EE** secures the channel **between sender and receiver** directly, so **no intermediate can read** the content; trust is needed only in the receiver, but the initial trust is hard to establish — e.g. PGP, S/MIME, Signal.
 ✓ **Likely graded:** P2PE = intermediates can read / trust the server, scalable (TLS); E2EE = only endpoints can read / trust only the receiver (PGP, S/MIME, Signal). [p.674] _____/2
- b) **Symmetric** alone: it requires a **pre-shared secret**, but there is no safe way to exchange that key over an insecure channel — key distribution is the core problem (and it scales badly, a key per pair). **Asymmetric** alone: it needs **no shared secret** and distributes keys easily, but the calculation is **expensive and slow**, unsuitable for bulk data. **Hybrid:** use the slow **asymmetric** cryptography only to exchange (or agree, via Diffie-Hellman) a **symmetric session key**, then use **fast symmetric** encryption for the actual data — combining easy key distribution with high performance.
 ✓ **Likely graded:** symmetric alone = unsolved key exchange / shared secret needed; asymmetric alone = too slow for bulk data; hybrid = asymmetric to exchange a symmetric session key, then symmetric for the payload. [p.631, 652, 656] _____/3
- c) ▶ The server's **asymmetric key pair** (**private key** plus **public key** in its X.509 certificate): authenticates the server and proves it owns the certificate, so Alice knows she is not talking to a man-in-the-middle. ▶ The **CA's keys** (root / intermediate): the certificate is **signed** by a trusted Certificate Authority, whose signature Alice verifies up the chain of trust. ▶ The **Diffie-Hellman (ephemeral) key shares** (DHE): exchanged to derive a fresh shared secret without ever transmitting it (perfect forward secrecy). ▶ The **symmetric session key** (e.g. AES_256): derived from the exchange and used for fast encryption of the actual traffic. ▶ A **MAC/HMAC key** (e.g. SHA384): keys the integrity check of each message. (Cipher suite TLS_DHE_AES_256_GCM_SHA384 names these parts.)
 ✓ **Likely graded:** server asymmetric key pair (authentication) + CA signing keys (chain of trust) + Diffie-Hellman shares (derive shared secret, PFS) + symmetric session key (bulk traffic) + MAC/HMAC key (integrity). [p.634, 641, 672, 681, 684] _____/4
- d) **Keys:** Bob's **public/private key pair**, Alice's **public/private key pair**, and a one-time **symmetric session key**. (1) **Speed:** hybrid encryption — the large attachment is encrypted **once** with the fast **symmetric session key**; only that small session key is encrypted asymmetrically, avoiding slow asymmetric operations on bulk data. (2) **Confidentiality:** the **session key** is encrypted with **Bob's public key**, so only **Bob's private key** can recover it and decrypt the message. (3) **Integrity + authenticity:** Alice computes a **hash** of the message and signs it with **her private key** (**digital signature**); Bob verifies it with **Alice's public key**, confirming the message is complete, unchanged and truly from Alice (non-repudiation). (4) **Arrival/read:** not guaranteed — the slides state that **receipt can be confirmed but not assured** (availability); cryptography cannot force delivery. A signed read-receipt could confirm reading but the recipient may refuse it (not on slides).
 ✓ **Likely graded:** keys (Bob's pair, Alice's pair, session key); (1) hybrid = symmetric for attachment, asymmetric only for the key; (2) session key encrypted with Bob's public key; (3) Alice signs a hash with her private key = digital signature; (4) delivery cannot be assured (receipt confirmed, not assured). [p.602, 643, 656, 663, 674] _____/6

Answer in the space below each question; fold along the line to check.

- a) Name three distinct types of factors that can be combined in **multi-factor authentication (MFA)**. Give an example for each. (3 P)
- b) Name the permission system that is used under Linux and macOS for authorizing local file accesses. Explain the main reason for this design decision. (3 P)
- c) (Mobile operating systems like iOS or Android describe permissions using **Capability Lists**.) Name concrete “elements” assigned to subject, object and permission in this context. (3 P)

ANSWERS (your score: _____ / 9 P)

 fold here

- a) Any combination of authentication factors from **different** categories (so one stolen factor is not enough):
1. **Something you know** — “**knowledge factors**”, e.g. password, PIN, security questions.
 2. **Something you have** — “**ownership factors**”, e.g. ID card, security token, cellphone (Mobile TANs / OTP).
 3. **Something you are** — “**inherence factors**” (biometric), e.g. fingerprint, retinal scan, voice.
- (The slides also list **location-based** and **time-based** factors.)
- ✓ **Likely graded:** three **distinct categories** — knowledge / ownership / inherence (not three passwords) — each with a valid example. [p.315] _____/3
- b) The **Access Control List (ACL)**. An ACL stores, **attached to each file/object**, who may do what — it is **the columns of the access control matrix** (permissions kept per object). Main reason: file systems are **object-centric** — the permissions live **in the file-system attributes next to each file**, which makes object permissions **easy to express and easy to change** per file (add/remove a file with its rights). On Linux it is implemented compactly as three rwx octets for **owner, group and other** (e.g. 750 = rwx r-x ---), giving a simple, maintainable per-file scheme.
- ✓ **Likely graded:** name **ACL**; reason = permissions stored per object (column of the matrix), inside the file-system attributes, making per-object permissions easy/efficient to set and change. [p.708] _____/3
- c) Mapping for the Android/iOS permission model:
- **Subject** — the **app** (the application / process requesting access).
 - **Object** — the protected **resource or data**, e.g. camera, microphone, location, contacts, media files, network.
 - **Permission** — the granted **capability / access right** on that resource, e.g. READ_CALENDAR, CAMERA, ACCESS_FINE_LOCATION (the action the app may perform on the object).
- ✓ **Likely graded:** subject = the app; object = the device resource / data (camera, contacts, location...); permission = the specific access right / capability granted on it. [p.741, 742] _____/3

Answer in the space below each question; fold along the line to check.

- d) Explain why capability lists are better suited in this scenario than the permission system of access control lists (Access Control Lists - ACLs). (3 P)
- e) (Android distinguishes between 1) “installation time” and 2) “runtime permission”.) Give an example for both types of permission. (2 P)
- f) Describe when and how the user decides about these permission grants. (2 P)

ANSWERS (your score: _____ / 7 P)

 fold here

- d) A **capability list** stores permissions **per subject** (the rows of the matrix) — exactly what a phone needs, since the natural question is “**what may THIS app do?**” and each app is checked/managed individually. Advantages that fit mobile:

- **Easy, efficient permissions per subject** — each app carries its own (small) set of capabilities.
- **Easy to change a subject’s permissions** — grant/revoke for one app without touching every object.
- **Works well with variable subjects** — apps are constantly installed/removed.
- **Decentral storage and control** — capabilities travel with the app.

ACLs are object-centric, so per-subject (per-app) management would be inefficient and hard to change here.

✓ **Likely graded:** capability list = per-subject (rows); mobile asks “what may this app do?”; cite easy/efficient per-subject perms, easy to change a subject, good with variable subjects, decentral control. [p.709, 710] _____/3

- e) 1. **Install-time permission** — granted automatically at install, low risk, e.g. **network/internet access** (or “run at startup”).
2. **Runtime permission** (“dangerous” permission) — e.g. **use the camera** or **access media files** / location.

✓ **Likely graded:** one valid install-time example (e.g. network access) and one runtime/“dangerous” example (e.g. camera, media files, location). [p.741] _____/2

- f) • **Install-time permissions:** the user effectively **accepts them up front when installing the app** — they are declared in the app’s **manifest** and granted automatically at install (no per-use prompt).
- **Runtime permissions:** the user is **asked at runtime, the first time the feature is used**, via a **system dialog** and explicitly allows or denies it (and can later change it in the permission manager / settings). This gives the user **control and transparency** (Implicit Deny — nothing without explicit allowance).

✓ **Likely graded:** install-time = accepted at install via the manifest declaration; runtime = explicit user confirmation in a system dialog when the feature is first used (revocable in settings). [p.741, 742] _____/2